

[Get Started](#)[Features](#)[SDK](#)[Changelogs ↗](#)[Blog ↗](#)

Master the workflows, prompt architectures, and local configuration choices to maximize agent velocity while maintaining robust control.

Establish verification loops

The single most effective way to ensure reliable, correct modifications from an autonomous agent is to provide the agent with a local verification mechanism (such as unit tests, build commands, or formatting scripts).

Before asking the agent to implement a code change:

1. Ensure your workspace directory has a test suite ready.
2. If tests do not exist, direct the agent to write a standard test block *first*.
3. Once the agent proposes code, instruct it to run the local test command to verify its work.
4. Watch the agent execute the command and iterate on the test outputs automatically.

```
> Implement feature X in main.py. Run npm test afterward to verify the build.
```



Explore, plan, then execute

Autonomous local agents operate with highest accuracy when complex changes are partitioned into distinct exploration, planning, and execution phases.

- **Exploration:** Ask the agent to explain how the target codebase resolves a particular problem or where an interface is defined before writing any changes.
- **Planning:** Request an implementation plan. The agent will list targeted files, required dependencies, and logic overrides in an implementation plan artifact.
- **Execution:** Once you approve the structured plan, direct the agent to apply the edits.

[Get Started](#)[Features](#)[SDK](#)[Changelogs ↗](#)[Blog ↗](#)

Enrich your prompting context

Give local agents high-fidelity indicators to narrow down reasoning boundaries and minimize token overhead.

Target file autocompletion

Type `@` within your prompt box to trigger the **Interactive Path Suggestion** overlay. Highlighting and selecting a path imports the absolute workspace file path directly into your prompt. This helps the agent target its code searches.

Attaching visual evidence

If debugging visual UI issues, rendering bugs, or frontend layout inconsistencies, capture a screenshot or video recording, copy it, and press `ctrl+v` inside the prompt box to attach it. The agent will consult the media file to diagnose the issue.

Configure your workspace environment

Optimize your local workstation rules and security boundaries to match your engineering flow.

Write a codebase rule file

Create a `GEMINI.md` or `AGENTS.md` file at your workspace root to outline specific directory standards, styling paradigms, test command parameters, and deprecation warnings. The agent automatically parses these rules on startup and consults them before suggesting changes.

Establish structured permissions

Tune your safety barriers in `~/.gemini/antigravity-cli/settings.json` based on your project risk level:

- **request-review** (Default): Prompts you before executing any write operations, bash commands, or remote network calls.

[Get Started](#)[Features](#)[SDK](#)[Changelogs ↗](#)[Blog ↗](#)

```
{  
  "toolPermission": "proceed-in-sandbox",  
  "enableTerminalSandbox": true  
}
```



Manage TUI sessions proactively

Use active session navigation tools to recover from engineering dead-ends or course-correct intermediate agent loops.

Course-correct early (esc)

If you watch an agent execute an incorrect search pattern or write code that deviates from your intentions, press the global escape hatch key `esc` immediately to interrupt the turn and regain focus of a clean prompt.

Rewind history with `/rewind`

If an agent has made several successive changes that introduce build errors, you do not need to discard the session. Type `/rewind` (or `/undo`) to roll back your conversation thread to a previous stable checkout.

Branch experiments with `/fork`

If you are unsure of the best implementation path:

1. Reach a stable baseline thread.
2. Type `/fork` to spin up a duplicate parallel session.
3. Test your speculative code modifications in the branched session.
4. If the approach fails, run `/resume` to swap back to your stable main branch.

[Get Started](#)[Features](#)[SDK](#)[Changelogs ↗](#)[Blog ↗](#)

Run non-interactive commands (-p)

To automate quick queries or integrate agents into git hooks, use the one-shot prompt flag `-p`:

```
agy -p "Review this git diff and draft a conventional commit message" --cwd  
$(pwd)
```



Fan out using parallel subagents

For large-scale sweeps or multi-file refactoring, direct the primary agent to spawn concurrent background subagents. The agent manager handles background threads autonomously while you continue working on your primary screen.

Related resources

Learn how to configure settings and customize visual layouts:

- **Settings, Rendering & Keybindings**: Customize keyboard hotkeys and buffers.
- **Permissions & Sandbox**: Enforce filesystem containment.
- **Plugins & Skills**: Create your own custom slash commands.

[← Features](#)[Troubleshooting >](#)